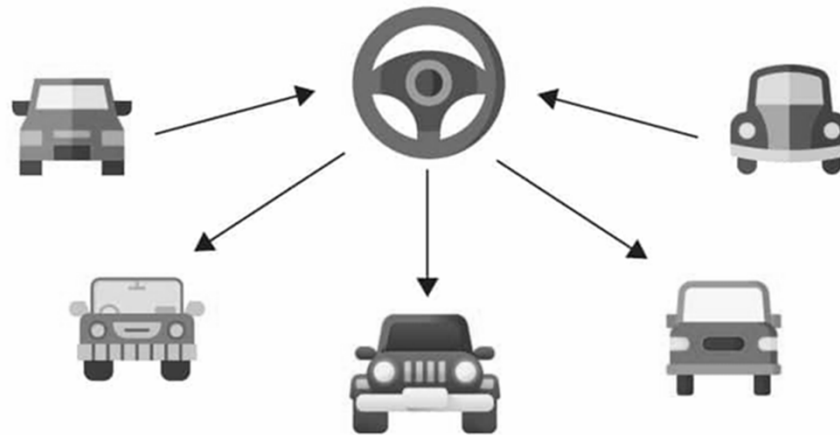


Object Oriented Programming

Polymorphism: many forms

The same word can have different meanings in different contexts — for example, “run” can mean jogging, running a program, or running a business.

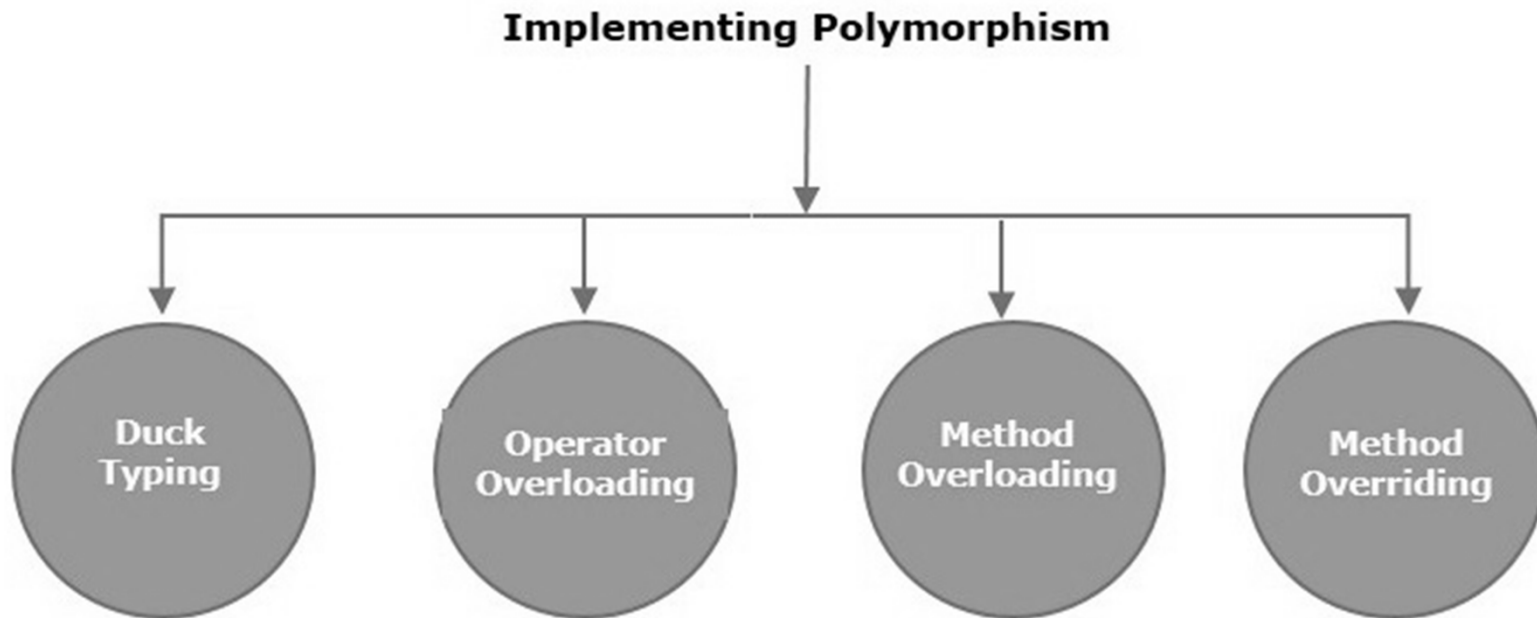


In programming, polymorphism means that the same method/function/operator behaves differently depending on the object or data.

Object Oriented Programming

Polymorphism:

Python supports four major forms of Polymorphism:



Object Oriented Programming

Method Overriding:

- Child class changes the parent class method implementation.
- Occur during the run time- Runtime Polymorphism

```
class A:  
    def show(self):  
        print("class A")
```

```
class B(A):  
    def show(self):  
        print("class B")
```

```
class C(A):  
    def show(self):  
        print("class C")
```

```
a=A()  
a.show()  
  
class A
```

```
a=B()  
a.show()  
  
class B
```

```
a=C()  
a.show()  
  
class C
```

Same Method → Different Output

Object Oriented Programming

Method Overloading:

Method overloading means defining multiple methods with the same name but with different parameters.

Examples:

- Add two numbers
- Add three numbers
- Add a list of number

Same Action $\longrightarrow$ Different Output

```
class math:
    def add(self,a,b):
        return a+b
    def add(self, a,b,c):
        return a+b+c
```

```
m=math()
m.add(1,2,3)
```

6

```
m=math()
m.add(1,2)
```

```
-----
TypeError
Cell In[7], line 2
      1 m=math()
----> 2 m.add(1,2)
```

Python does not support true method overloading because:

- Dynamically typed
- Interpreted at run time
- No compile-time method resolution

Object Oriented Programming

How Python achieves Method Overloading:

- Default arguments
- * args (Variable positional argument)
- ** kwargs (variable Keyword arguments)

Default argument:

```
class math:  
    def add (self,a,b=0,c=0):  
        return a+b+c  
  
m=math()
```

```
m.add(2,3)
```

5

```
m.add(1,2,3)
```

6

Object Oriented Programming

How Python achieves Method Overloading:

Using *args:

```
class math:
    def add (self,*args):
        total=0
        for i in args:
            total=total+i
        return total
```

```
m=math()
```

```
m.add(1,2,3)
```

6

```
m.add(2,4,6,8)
```

20

Using **kwargs:

```
class student:
    def info(self,**kwargs):
        for key, value in kwargs.items():
            print(key,":", value)
```

```
s=student()
```

```
s.info(name="Rahul")
```

name : Rahul

```
s.info(name="Ram",age=21)
```

name : Ram

age : 21

Object Oriented Programming

Operator Overloading:

- Python operators work for built-in classes in a predefined operational way, but the same operator behaves differently for objects of different classes.
- Example:
 - + operator
 - Arithmetic addition of two numbers
 - Merge two lists
 - Concatenate two strings
- This feature in Python that allows the same operator to have different meanings according to the context is called operator overloading.

Object Oriented Programming

Operator Overloading:

```
class point:  
    def __init__(self,x,y):  
        self.x=x  
        self.y=y
```

```
p1=point(2,3)  
p2=point(4,5)
```

```
print(p1+p2)
```

```
-----  
TypeError  
Cell In[25], line 1  
----> 1 print(p1+p2)
```

Because the computer doesn't know how to add two objects

```
class point:  
    def __init__(self,x,y):  
        self.x=x  
        self.y=y  
  
    def __add__(self,other):  
        return point(self.x+other.x,self.y+other.y)  
  
    def __str__(self):  
        return f"({self.x}, {self.y})"
```

```
: p1=point(2,3)  
   p2=point(4,5)
```

```
print(p1+p2)  
  
(6, 8)
```

We will define a method for an operator, and that process is called operator overloading

Object Oriented Programming

Operator Overloading:

Python converts operators into method calls.

Built-In Method Name	Operator
<code>__add__()</code>	<code>+</code> (add)
<code>__sub__()</code>	<code>-</code> (minus)
<code>__mul__()</code>	<code>*</code> (multiply)
<code>__div__()</code>	<code>/</code> (divide)
<code>__mod__()</code>	<code>%</code> (remainder/modulo)
<code>__pow__()</code>	<code>**</code> (power)
<code>__lt__()</code>	<code><</code> (less than)
<code>__gt__()</code>	<code>></code> (greater than)
<code>__le__()</code>	<code><=</code> (less than or equal to)
<code>__ge__()</code>	<code>>=</code> (greater than or equal to)
<code>__eq__()</code>	<code>==</code> (equality check)
<code>__ne__()</code>	<code>!=</code> (inequality check)

Object Oriented Programming

Duck Typing:

```
class Dog:
    def speak(self):
        print("Woof")
class Cat:
    def speak(self):
        print("Meow")
def make_sound(animal):
    if isinstance(animal, Dog):
        animal.speak()
    elif isinstance(animal, Cat):
        animal.speak()

make_sound(Dog())
make_sound(Cat())
```

Woof
Meow

```
class Dog:
    def speak(self):
        print("Woof")
class Cat:
    def speak(self):
        print("Meow")
class Cow:
    def speak(self):
        print("Moo")

def make_sound(animal):
    if isinstance(animal, Dog):
        animal.speak()
    elif isinstance(animal, Cat):
        animal.speak()

make_sound(Dog())
make_sound(Cat())
```

```
class Dog:
    def speak(self):
        print("Woof")
class Cat:
    def speak(self):
        print("Meow")
class Cow:
    def speak(self):
        print("Moo")

def make_sound(animal):
    if isinstance(animal, Dog):
        animal.speak()
    elif isinstance(animal, Cat):
        animal.speak()
    elif isinstance(animal, Cow):
        animal.speak()

make_sound(Dog())
make_sound(Cat())
make_sound(Cow())
```

Woof
Meow
Moo

Duck typing is a concept in Python where the **type/class** of an object is less important than what the object can do.

Object Oriented Programming

Duck Typing:

- Duck typing is a concept related to polymorphism in Python that focuses on an object's behavior (methods/properties) rather than its actual type.
- It comes from the phrase:
"If it walks like a duck and quacks like a duck, it's a duck."

```
class Duck:
    def speak(self):
        return "Quack"
class Dog:
    def speak(self):
        return "Bark"
class Cat:
    def speak(self):
        return "Meow"

def animal_sound(animal):
    print(animal.speak())
```

```
d=Duck()
dog=Dog()
c=Cat()
```

```
: animal_sound(d)
  animal_sound(dog)
  animal_sound(c)
```

```
Quack
Bark
Meow
```

The function `animal_sound()` does not care about the object type; it only expects the object to have a `.speak()` method.

Type is less important than behavior
OR
behavior matters more than its type.

Exercise

Create a Python program to manage a school system, demonstrating Object-Oriented Programming concepts such as Method Overloading, Method Overriding, Duck Typing, and Operator Overloading.

Requirements:

- Create a base class Person with attributes:
 - name (string)
 - age (integer)
 - Method display() to print name and age.
- Create a subclass Teacher that inherits from Person:
 - Adds an attribute subject
 - Overrides display() to include the subject.
- Create another subclass Student that inherits from Person:
 - Adds attributes student_id and grade
 - Overrides display() to include these details.
 - Create a method add_marks() that allows adding marks for one or multiple subjects

Exercise

Create a Python program to manage a school system, demonstrating Object-Oriented Programming concepts such as Method Overloading, Method Overriding, Duck Typing, and Operator Overloading.

Requirements:

- Create a further subclass ExamResult that inherits from Student:
 - Adds a method display_result() to print all student details including marks.
 - Overload the + operator using __add__() to calculate the total marks of two students.
- Create a function show_info(obj) that calls the display() method.
 - Pass the Teacher and Student Object to this function
 - The function should work for both because they both have a display method.
- Create objects for at least one teacher and one student with exam results, and display their information.

Exercise

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)
```

```
class Teacher(Person):
    def __init__(self, name, age, subject):
        super().__init__(name, age)
        self.subject = subject

    def display(self):
        super().display()
        print("Subject:", self.subject)
```

```
class Student(Person):
    def __init__(self, name, age, student_id, grade):
        super().__init__(name, age)
        self.student_id = student_id
        self.grade = grade
        self.marks = {}

    def display(self):
        super().display()
        print("Student ID:", self.student_id)
        print("Grade:", self.grade)

    def add_marks(self, *args):
        for subject, mark in args:
            self.marks[subject] = mark
```

```
class ExamResult(Student):
    def display_result(self):
        self.display()
        print("Marks:")
        for subject, score in self.marks.items():
            print(subject, ":", score)

    def __add__(self, other):
        total1 = sum(self.marks.values())
        total2 = sum(other.marks.values())
        return total1 + total2
```

```
def show_info(obj):
    obj.display()
```

Exercise

```
# Creating objects
teacher = Teacher("Mr. Sharma", 40, "Mathematics")

student1 = ExamResult("Rahul", 16, "S101", "10th")
student2 = ExamResult("Anita", 17, "S102", "10th")

# Method Overloading usage
student1.add_marks(("Math", 90), ("Science", 85), ("English", 88))
student2.add_marks(("Math", 80), ("Science", 75), ("English", 82))

# Duck Typing
print("Teacher Information:")
show_info(teacher)

print("\nStudent Information:")
show_info(student1)

# Display results
print("\nExam Result of Student 1:")
student1.display_result()

print("\nExam Result of Student 2:")
student2.display_result()

# Operator Overloading
total_marks = student1 + student2
print("\nTotal Marks of both students:", total_marks)
```

Teacher Information:

Name: Mr. Sharma

Age: 40

Subject: Mathematics

Student Information:

Name: Rahul

Age: 16

Student ID: S101

Grade: 10th

Exam Result of Student 1:

Name: Rahul

Age: 16

Student ID: S101

Grade: 10th

Marks:

Math : 90

Science : 85

English : 88

Exam Result of Student 2:

Name: Anita

Age: 17

Student ID: S102

Grade: 10th

Marks:

Math : 80

Science : 75

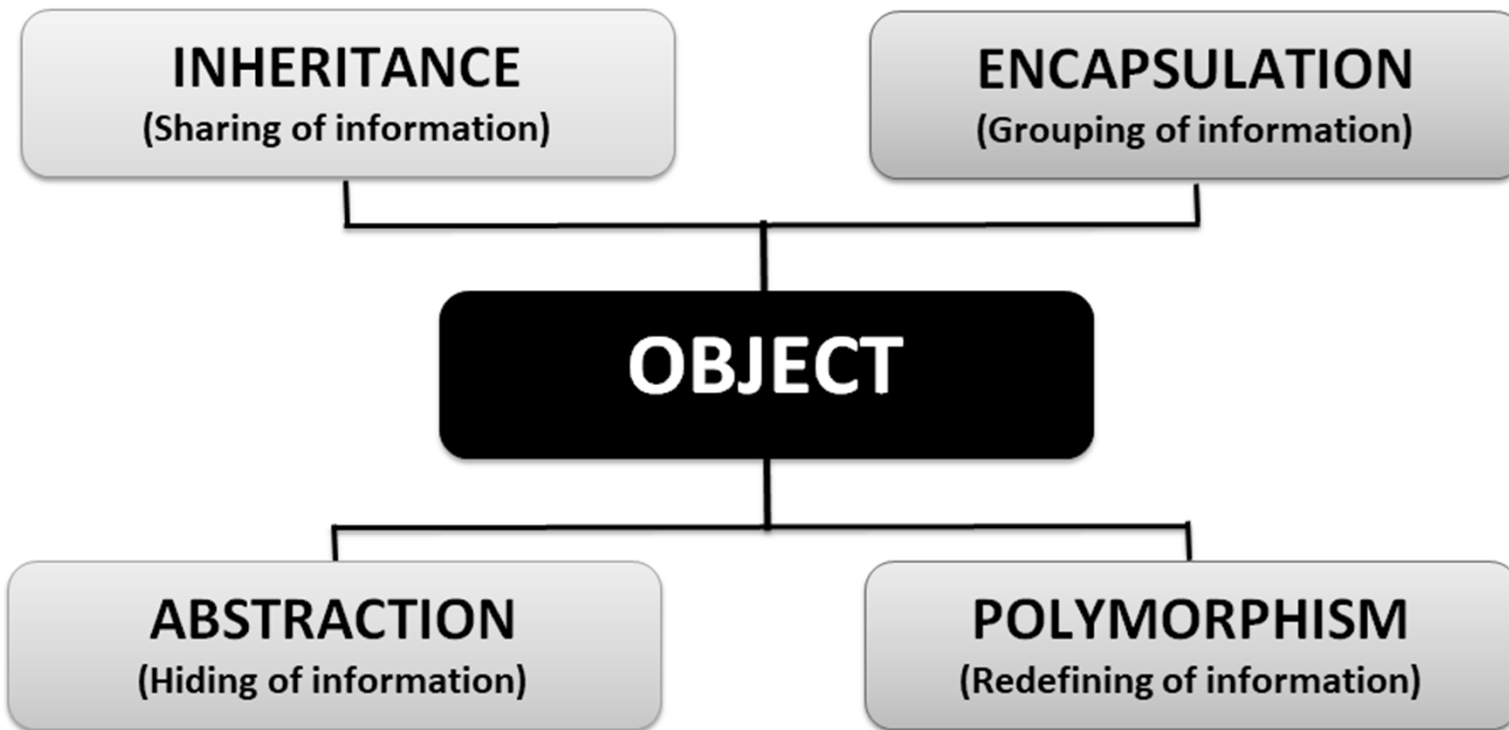
English : 82

Total Marks of both students: 500

194

Object Oriented Programming

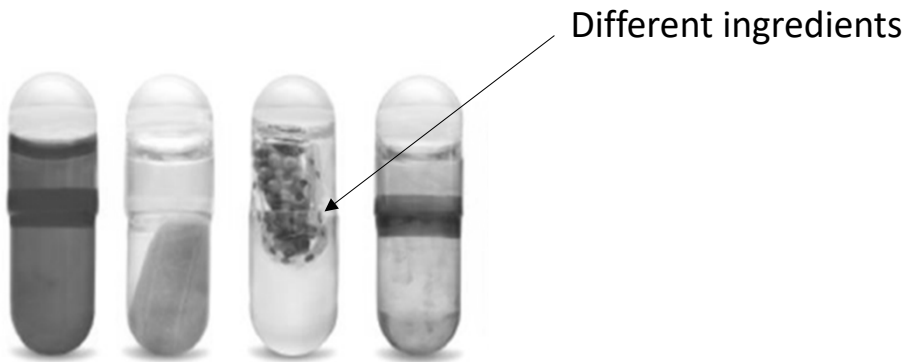
Four key principles of the OOPS:



Object Oriented Programming

Encapsulation:

Encapsulation is about bundling data (variables) and methods (functions) into a single unit (class) and restricting direct access to some of the object's components.



Encapsulation is the process of wrapping data and methods together in a class and restricting direct access to the data.

It helps in:

- Protects data from unauthorized access
- Prevents accidental modification
- Improves code security
- Makes code modular and maintainable

Similarly, in programming, you hide the inner details of how data is stored or changed, and you give controlled access through methods

Object Oriented Programming

Types/Access modifiers:

Public

```
class Student:
    def __init__(self):
        self.name = "Rahul"

s = Student()
print(s.name)
```

Rahul

Accessible everywhere:
inside or outside the class

Private

```
: class Student:
    def __init__(self):
        self.__marks = 90

s = Student()
print(s.__marks)
```

```
-----
AttributeError
Cell In[6], line 6
      3         self.__marks = 90
      5 s = Student()
----> 6 print(s.__marks)
```

```
: class Student:
    def __init__(self):
        self.__marks=90

s=Student()

print(s._Student__marks)

90
```

Name Mangling

Protected

```
: class Student:
    def __init__(self):
        self._age = 20

s = Student()
print(s._age)
```

20

- Single underscore _
- Accessible but discouraged
- Used in inheritance

Member	Same Class	Child Class	Outside
Public	✓	✓	✓
Protected	✓	✓	⚠
Private	✓	✗	✗

Object Oriented Programming

Accessing Private Attributes (getter and setter methods):

```
: class Student:
    def __init__(self):
        self.__marks = 90

s = Student()
print(s.__marks)
```

```
-----
AttributeError
Cell In[6], line 6
      3         self.__marks = 90
      5 s = Student()
----> 6 print(s.__marks)
```

A private attribute can be assessed using getter methods

```
class Student:

    def __init__(self, age):
        self.__age = age

    def get_age(self):
        return self.__age

s = Student(20)
print(s.get_age())
```

20

Object Oriented Programming

Accessing Private Attributes (getter and setter methods):

```
class Student:
    def __init__(self, age):
        self.age=age

s=Student(20)
print(s.age)
```

20

```
s.age=25
print(s.age)
```

25

```
class Student:
    def __init__(self, age):
        self.__age=age

s=Student(20)

s.__age=25
print(s.__age)
```

25

```
class Student:
    def __init__(self, age):
        self.__age=age

s=Student(20)
s.__age=25
print(s.__age)

print(s._Student__age)
```

25

20

A private attribute can be modified using setter methods

```
class Student:

    def __init__(self, age):
        self.__age = age

    def get_age(self):
        return self.__age

    def set_age(self, age):
        if age > 0:
            self.__age = age
        else:
            print("Invalid age")

s = Student(20)

print(s.get_age())
s.set_age(25)
print(s.get_age())
```

20

25

Object Oriented Programming

Encapsulation with Inheritance

Encapsulation with inheritance means hiding data in a parent class and allowing controlled access to child classes based on access modifiers.

```
class BankAccount:
    def __init__(self, balance):
        self.balance = balance
    def deposit(self, amount):
        self.balance += amount

class SavingsAccount(BankAccount):
    def add_interest(self):
        interest = self.balance * 0.05
        self.deposit(interest)

acc = SavingsAccount(1000)
acc.add_interest()
print("Balance:", acc.balance)
```

Balance: 1050.0

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance    # Private
    def deposit(self, amount):
        self.__balance += amount
    def get_balance(self):
        return self.__balance

class SavingsAccount(BankAccount):
    def add_interest(self):
        interest = self.get_balance() * 0.05
        self.deposit(interest)
```

```
acc = SavingsAccount(1000)
acc.add_interest()
print("Balance:", acc.get_balance())
```

Balance: 1050.0

Object Oriented Programming

Protected

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance    # Private data
    # Private method
    def __show_balance(self):
        print("Balance is:", self.__balance)
    # Public method to access private method
    def get_balance(self):
        #self.__show_balance()
        return self.__balance
    # Public deposit method
    def deposit(self, amount):
        self.__balance += amount

class SavingsAccount(BankAccount):
    def add_interest(self):
        interest = self.get_balance() * 0.05
        self.deposit(interest)
        self.__show_balance()

acc = SavingsAccount(1000)
acc.add_interest()
print("Final Balance:", acc.get_balance())
```

Balance is: 1050.0
Final Balance: 1050.0

Private

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance    # Private data
    # Private method
    def __show_balance(self):
        print("Balance is:", self.__balance)
    # Public method to access private method
    def get_balance(self):
        #self.__show_balance()
        return self.__balance
    def access_balance(self):
        return self.__show_balance()
    # Public deposit method
    def deposit(self, amount):
        self.__balance += amount

class SavingsAccount(BankAccount):
    def add_interest(self):
        interest = self.get_balance() * 0.05
        self.deposit(interest)
        self.access_balance()

acc = SavingsAccount(1000)
acc.add_interest()
print("Final Balance:", acc.get_balance())
```

Balance is: 1050.0
Final Balance: 1050.0

Exercise

A bank wants to create a system where:

- Customer balance should not be directly accessible.
- Users can only:
 - Deposit money
 - Withdraw money
 - Check balance
 - Balance should never become negative

```
acc = BankAccount(1000)

acc.deposit(500)      # Deposit
acc.withdraw(300)     # Withdraw
acc.withdraw(1500)    # Exceeds balance

print("Current Balance:", acc.get_balance())
```

```
class BankAccount:
    def __init__(self, balance):
        # Private variable
        self.__balance = balance
    # Deposit method
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print("Deposited:", amount)
        else:
            print("Invalid deposit amount")
    # Withdraw method
    def withdraw(self, amount):
        if amount <= 0:
            print("Invalid withdrawal amount")
        elif amount > self.__balance:
            print("Insufficient balance")
        else:
            self.__balance -= amount
            print("Withdrawn:", amount)
    # Check balance method (Getter)
    def get_balance(self):
        return self.__balance
```

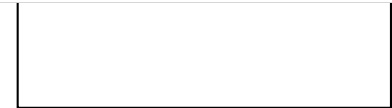
Object Oriented Programming

Abstraction:

- Abstraction is the process of hiding the internal details and showing only the functionality.
- It helps reduce complexity and allows focus on what the object does rather than how it does it.
- In Python, abstraction is used to hide irrelevant data or classes to reduce complexity. It also enhances the application efficiency.
- It is essential to hide the core functionality from the user.
- **In Python, abstraction can be achieved by abstract classes and abstract methods.**

Object Oriented Programming

Abstraction:



- Abstract classes are classes that contain one or more abstract methods.
- We cannot create an object of an abstract class.
- An abstract method is a method that is declared, but contains no implementation. (in C++ or Java, but in Python, abstract methods CAN have implementation.)
 def abs():
 pass
- Abstract classes cannot be instantiated and require subclasses to provide the implementations for the abstract methods.
- We can create abstract base classes in Python using abc.

Object Oriented Programming

Abstraction:

```
: from abc import ABC, abstractmethod
```

```
class Image(ABC):
```

```
    @abstractmethod
```

```
    def area(self,a,b=0):  
        pass
```

```
class Rectangle(Image):
```

```
    def ok(self):  
        pass
```

```
: r1=Rectangle()
```

Abstract method is not defined in the subclass

```
r1=Rectangle()
```

```
-----  
TypeError
```

```
Traceback (most recent call last)
```

```
Cell In[2], line 1
```

```
----> 1 r1=Rectangle()
```

```
TypeError: Can't instantiate abstract class Rectangle without an implementation for abstract method 'area'
```

Object Oriented Programming

Abstraction:



```
: from abc import ABC, abstractmethod

class Image(ABC):

    @abstractmethod
    def area(self,a,b=0):
        pass

class Rectangle(Image):
    def ok(self):
        pass
    def area(self,a,b=0):
        c=a*b
        return c
```

Create an object of the abstract class

```
: r1=Image()
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[4], line 1
----> 1 r1=Image()

TypeError: Can't instantiate abstract class Image without an implementation for abstract method 'area'
```

Object Oriented Programming

Abstraction:

```
from abc import ABC, abstractmethod
```

```
class Image(ABC):
```

```
    @abstractmethod
    def area(self,a,b=0):
        pass
```

```
class Rectangle(Image):
    def ok(self):
        pass
    def area(self,a,b=0):
        c=a*b
        return c
```

```
r1=Rectangle()
```

```
print(r1.area(2,4))
```

```
from abc import ABC, abstractmethod
```

```
class Image(ABC):
```

```
    @abstractmethod
    def area(self,a,b=0):
        pass
```

```
class Rectangle(Image):
```

```
    def area(self,a,b=0):
        c=a*b
        return c
```

```
class Circle(Image):
```

```
    def area(self,r):
        c=3.14*r*r
        return c
```

```
c1=Circle()
print(c1.area(10))
```

```
314.0
```

Object Oriented Programming

Abstraction:

Constructor in Abstract class

```
from abc import ABC, abstractmethod

class Image(ABC):
    def __init__(self):
        print("Hello")
    @abstractmethod
    def area(self,a,b=0):
        pass

class Rectangle(Image):
    def area(self,a,b=0):
        c=a*b
        return c

class Circle(Image):
    def area(self,r):
        c=3.14*r*r
        return c
```

```
c1=Circle()
```

Hello

Constructor in Child class

```
from abc import ABC, abstractmethod

class Image(ABC):
    def __init__(self):
        print("Hello")
    @abstractmethod
    def area(self,a,b=0):
        pass

class Rectangle(Image):
    def __init__(self):
        print("Hi")
    def area(self,a,b=0):
        c=a*b
        return c

class Circle(Image):
    def area(self,r):
        c=3.14*r*r
        return c
```

```
c1=Circle()
```

Hello

```
r1=Rectangle()
```

Hi

Exercise

Design an Online Shopping System using Object Oriented Programming concepts.

Requirements:

- Create an abstract class Product
- Product should have:
 - name
 - price
 - abstract method calculate_discount()
- Create two subclasses:
 - Electronics
 - Clothing
- Apply different discount rules:
 - Electronics → 10% discount
 - Clothing → 20% discount
- Create objects and print the final price after the discount.

Add a constructor, inheritance, polymorphism, encapsulation, and abstraction in the Online Shopping System using OOP concepts.